



**S. B. JAIN INSTITUTE OF TECHNOLOGY,
MANAGEMENT AND RESEARCH, NAGPUR**

(An Autonomous Institute Affiliated to R.T.M. Nagpur University, Nagpur)

**DEPARTMENT OF ELECTRONICS AND
TELECOMMUNICATION ENGINEERING**

(NBA Accredited)

Emerge as a center for quality education in Electronics and Telecommunication Engineering, so as to create competent professionals



Academic Session 2025-26 (EVEN)

Teachers Assessment Examination-I:

Year/Semester: III/VI

Course: IoT and Network Protocols (N-MDMIT601T)

Name of Student:	Gaurav hawelkar		
USN No.	CS23026	Date of Submission: 25/03/26	

***Final Date of Submission:**

Problem Statement	Date of Execution
Stage I Identification of IoT components and required components.	
Stage II Designing and implementation.	

Evaluation Report	Stage I	Stage II	Stage III	Total
Marks Obtained				
Marks out of	02	02	01	05

Name and Signature of Course In-charge:

1. Title: Smart Parking System

2. Problem Statement:

Build a smart parking sensor system that detects car presence using ultrasonic sensors. Pico W should update a central database or app to show parking availability in real time.

3. Working Principle

The system works based on IoT (Internet of Things):

1. Sensors detect parking status:
 - o Ultrasonic Sensor → Distance from vehicle
 - o Determines presence/absence of a car
2. Raspberry Pi Pico W reads sensor data.
3. Data is processed:
 - o Distance is calculated from echo time
 - o Converted into slot status (Occupied / Vacant)
4. The Pico W sends data to the cloud using Wi-Fi.
5. Cloud platform (e.g., ThingSpeak / Firebase) stores and updates parking data.
6. A web/mobile dashboard displays:
 - o Live parking availability
 - o Slot-wise status (Free / Occupied)
7. Indicators show real-time status:
 - o Slot occupied → LED ON (Red)
 - o Slot vacant → LED ON (Green)

4. Procedure

Step 1: Collect all required components (Pico W, ultrasonic sensor, LEDs, resistors, wires, breadboard).

Step 2: Connect the ultrasonic sensor to the Pico W (VCC, GND, TRIG, ECHO pins).

Step 3: Connect red and green LEDs to GPIO pins using resistors.

Step 4: Install MicroPython and set up the coding environment (e.g., Thonny IDE).

Step 5: Write code to trigger the ultrasonic sensor and measure distance.

Step 6: Set a threshold value to identify parking status (Occupied/Vacant).

Step 7: Program LEDs to indicate status (Red = Occupied, Green = Vacant).

Step 8: Configure Wi-Fi on Pico W using SSID and password.

Step 9: Send parking data to a cloud platform (ThingSpeak/Firebase).

Step 10: Create a dashboard to display real-time parking availability.

Step 11: Test the system and adjust the sensor threshold if needed.

Step 12: Run the system continuously for real-time monitoring.

5. Components Required

Sr. No.	Component Name	Purpose
1	Raspberry Pi Pico W	Main microcontroller with Wi-Fi
2	Ultrasonic Sensor (HC-SR04)	Detects car presence (distance)
3	LED (Red)	Indicates occupied slot
4	LED (Green)	Indicates vacant slot
5	Resistors (220Ω / 330Ω)	Protect LEDs from high current
6	Breadboard	Circuit assembly
7	Jumper Wires	Electrical connections
8	USB Cable / Power Supply	Powers the Pico W
9	Wi-Fi Network	Enables data transmission
10	Cloud Platform (ThingSpeak/Firebase)	Stores & displays data

6. Code:

Raspberry Pi pico Code:

```
from machine import Pin, UART
import time
```

```
# Pins
```

```
ir = Pin(15, Pin.IN)      # IR sensor OUT
```

```
green = Pin(16, Pin.OUT)  # Green LED
```

```
red = Pin(17, Pin.OUT)    # Red LED
```

```
buzzer = Pin(18, Pin.OUT) # Buzzer
```

```
# UART (to ESP8266)
```

```
uart = UART(0, baudrate=9600, tx=Pin(0), rx=Pin(1))
```

```
# Parking
```

```
count = 0
```

```
capacity = 10
```

```
last_state = 1 # for detection
```

```
while True:
```

```
    current_state = ir.value()
```

```
    # Detect car (falling edge)
```

```
if last_state == 1 and current_state == 0:
    if count < capacity:
        count += 1
        print("Car Entered:", count)
```

```
    # Send to ESP
    uart.write(str(count) + "\n")
```

```
    time.sleep(1) # debounce
```

```
last_state = current_state
```

```
# LED + BUZZER LOGIC
```

```
if count >= capacity:
    red.value(1)
    green.value(0)
    buzzer.value(1) # ON
```

```
else:
    red.value(0)
    green.value(1)
    buzzer.value(0)
```

```
time.sleep(0.1)
```

ESP8266 Code:

```
#include <ESP8266WiFi.h>
```

```
#include <ESP8266HttpClient.h>
```

```
// WiFi
```

```
#define WIFI_SSID "realme"
```

```
#define WIFI_PASSWORD "user@12345"
```

```
// Supabase (UPDATED)
```

```
const char* supabaseUrl = "https://czggojwcsasptljblzfe.supabase.co/rest/v1/parking?id=eq.1";
```

```
const char* apiKey = "sb_publishable_M8_LLmMGaeCzsXOzUFar-w_aDPgMrXo";
```

```
// Parking
```

```
int count = 0;
```

```
int capacity = 10;
```

```
void setup() {
```

```
    Serial.begin(9600); // match Pico baud rate
```

```
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
```

```
    Serial.print("Connecting to WiFi");
```

```
    while (WiFi.status() != WL_CONNECTED) {
```

```
        delay(500);
```

```
        Serial.print(".");
```

```
    }
```

```

Serial.println("\nWiFi Connected!");
}

void loop() {

// Receive data from Pico
if (Serial.available()) {
    String data = Serial.readStringUntil('\n');
    count = data.toInt();

    Serial.println("Received from Pico: " + String(count));
}

// 📶 Send data to Supabase
if (WiFi.status() == WL_CONNECTED) {

    WiFiClient client;
    HTTPClient http;

    http.begin(client, supabaseUrl);

    http.addHeader("Content-Type", "application/json");
    http.addHeader("apikey", apiKey);
    http.addHeader("Authorization", String("Bearer ") + apiKey);

    String status = (count >= capacity) ? "full" : "available";

    String json = "{\"count\": " + String(count) + ", \"status\": \"" + status + "\"}";

    int response = http.PATCH(json);

    Serial.println("Sent to Supabase: " + String(response));

    http.end();
}

delay(2000);
}

```

Dashboard Code:

```
import React, { useState, useEffect, useRef } from "react";
import { CarFront, LogIn, LogOut, RotateCcw, Activity, AlertTriangle, Wifi, WifiOff } from "lucide-react";
import { clsx } from "clsx";
import { twMerge } from "tailwind-merge";

// Utility for Tailwind classes
export function cn(...inputs) {
  return twMerge(clsx(inputs));
}

export default function App() {
  const MAX_CAPACITY = 10;

  const [count, setCount] = useState(0);
  const [status, setStatus] = useState("available");
  const [logs, setLogs] = useState([]);
  const [showResetConfirm, setShowResetConfirm] = useState(false);
  const [sensorConnected, setSensorConnected] = useState(true);

  const prevCountRef = useRef(0);
  const supabaseUrl = "https://czggojwcsasptljblzfe.supabase.co/rest/v1/parking?id=eq.1";
  const apiKey = "sb_publishable_M8_LLmMGaeCzsXOzUFar-w_aDPgMrXo";

  useEffect(() => {
    const fetchData = async () => {
      try {
        const res = await fetch(supabaseUrl, {
          headers: { apikey: apiKey }
        });
        const data = await res.json();

        if (data && data.length > 0) {
          const newCount = data[0].count;
          const newStatus = data[0].status;

          setStatus(newStatus || (newCount >= MAX_CAPACITY ? "Parking Full" : "available"));

          if (newCount !== prevCountRef.current) {
            const time = new Date().toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' });

            let actionLog = "";
            if (newCount > prevCountRef.current) {
              actionLog = `Car entered: ${newCount}`;
            } else if (newCount < prevCountRef.current) {
              actionLog = `Car exited: ${newCount}`;
            }

            setLogs(prev => {
              const newLogs = actionLog ? [{ action: actionLog, time }, ...prev].slice(0, 5) : prev;
              if (newCount >= MAX_CAPACITY && prevCountRef.current < MAX_CAPACITY) {

```

```

        return [{ action: "Parking Full", time }, ...newLogs].slice(0, 5);
    }
    return newLogs;
});

    setCount(newCount);
    prevCountRef.current = newCount;
}
}
} catch (err) {
    console.error("Error fetching data from Supabase:", err);
}
};

fetchData(); // Initial fetch
const interval = setInterval(fetchData, 1500); // Poll every 1.5 seconds

return () => clearInterval(interval);
}, []);

const available = MAX_CAPACITY - count;
const isFull = count >= MAX_CAPACITY;
const isEmpty = count === 0;
const isAlmostFull = count >= 8 && count < 10;

const updateSupabase = async (newCount) => {
    const newStatus = newCount >= MAX_CAPACITY ? "Parking Full" : "available";
    try {
        await fetch(supabaseUrl, {
            method: "PATCH",
            headers: {
                "Content-Type": "application/json",
                apikey: apiKey,
                Authorization: `Bearer ${apiKey}`
            },
            body: JSON.stringify({
                count: newCount,
                status: newStatus
            })
        });
    } catch (err) {
        console.error("Failed to update Supabase", err);
    }
};

// Updating optimistically for instant UI feedback
setCount(newCount);
setStatus(newStatus);
prevCountRef.current = newCount;

const handleEntry = () => {
    if (count < MAX_CAPACITY) {
        const newCount = count + 1;
        const time = new Date().toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' });
    }
};

```

```

    setLogs(prev => [{ action: `Car entered: ${newCount}`, time }, ...prev].slice(0, 5));
    if (newCount >= MAX_CAPACITY) {
      setLogs(prev => [{ action: "Parking Full", time }, ...prev].slice(0, 5));
    }
    updateSupabase(newCount);
  }
};

```

```

const handleExit = () => {
  if (count > 0) {
    const newCount = count - 1;
    const time = new Date().toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' });
    setLogs(prev => [{ action: `Car exited: ${newCount}`, time }, ...prev].slice(0, 5));
    updateSupabase(newCount);
  }
};

```

```

const confirmReset = async () => {
  try {
    await fetch(supabaseUrl, {
      method: "PATCH",
      headers: {
        "Content-Type": "application/json",
        apiKey: apiKey,
        Authorization: `Bearer ${apiKey}`
      },
      body: JSON.stringify({
        count: 0,
        status: "available"
      })
    });
  }
};

```

```

const time = new Date().toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' });
setLogs([{ action: "System Reset", time }]);

setCount(0);
setStatus("available");
prevCountRef.current = 0;
setShowResetConfirm(false);
} catch (err) {
  console.error(err);
}
};

```

7. Result:

The smart parking system successfully detects the presence of a vehicle using an ultrasonic sensor. The Raspberry pi Pico W accurately processes the distance data and determines whether a parking slot is occupied or vacant. The system updates the parking status in real time to a cloud platform via Wi-Fi. The web/mobile dashboard displays live parking availability, and LEDs provide clear on-site indication of slot status. The system operates reliably with continuous monitoring and quick response.

8. Conclusion

The Smart Parking System developed in this project demonstrates the effective use of IoT technology in modern parking management. By integrating an ultrasonic sensor with the Raspberry Pi Pico W, the system enables continuous and real-time monitoring of parking space availability.

The collected data is transmitted to a cloud platform such as ThingSpeak or Firebase, where it is stored and visualized through dashboards and status displays. This allows users to easily check parking availability remotely, saving time and effort.

The system also includes LED indicators, which provide instant visual feedback about whether a parking slot is occupied or vacant, making it practical even without accessing the dashboard. The implementation proves that such a system can be built at low cost with simple hardware while delivering efficient results.

Overall, this project highlights how smart parking solutions can:

- Reduce time spent searching for parking
- Minimize traffic congestion
- Improve space utilization
- Enable real-time and data-driven parking management

9. Geotag:



